# Izmir Institute of Technology

## Electrical and Electronics Eng. Dep.

# EE 471 Modern Software Development Practices and Technologies

# In-class exercise Week #3

Monday 12/10/2026

**Preliminary Task:** Experiment on OOP -> [1] and [2]; clean&vibe coding -> [3] and [4].

**Part 1: Vibe coding & Clean coding integrated**

1) Install Github Copilot, Claude code, and Codex extensions in VSCode. Download and install Antigravity and Cursor on your system.

2) Using each of the platforms, try to improve the “conform.py” script in terms of clean coding practices. Try out different LLM models (e.g. Gemini 3.1 or Claude Sonnet 4.6 etc.) and compare their performance.

**Deliverable:** Report a 1-paragraph comparison of the improvements in the revised script (2 platforms, one model each). Comment on your experience with different platforms and different LLMs. Saved as “part1.txt”.

**Part 2: Foundations of Classes**

1) Create a folder Week3 and work inside this folder. Create a Python script “custom_classes.py” and create a base class `MediaItem` that represents a generic item in a library.

- **Attributes:** Initialize with `title`, `author`, and `is_available` (boolean, default True).
- **Methods:** `checkout()`: If available, set `is_available` to False and return a success message “Successful checkout”. If not, return "Already out".
- `return_item()`: Set `is_available` to True.
- `__str__`: Override this to return: "Title by Author [Available/Checked Out]"

2) Create a python script “main.py”. Inside this script, create two `MediaItem` instances (a book and a movie) and simulate checking one out. Push Week3 to your Git Repo, committed as “part2”.
**Deliverable:** A screen recording video you simulate checking one item out; saved as “part2.avi”.

**Part 3: Inheritance & Hierarchies**

Extend the system using inheritance. Libraries don't just have generic items; they have specific rules for different media.

1) Inside “custom_classes.py”, create a subclass `Book`, and add an attribute of `page_count`. Override `__str__` to include the page count.
2) Inside “custom_classes.py”, create a subclass `DVD`, and add an attribute of `duration`. Override `checkout()`: Use `super().checkout()` to maintain the original checkout logic and preserve its return value. DVDs are fragile. Every time it is successfully checked out, it should print a reminder: *"Handle with care: Do not scratch the disc."*
3) Inside “main.py”, create a DVD instance and simulate checking it out.
4) Push Week3 folder to your Git Repo, committed as “part3”.

**Deliverables:** A screen recording video you simulate checking DVD item out; saved as “part3.avi”. What is the use of `super()` here ? What happens if we change the logic in `MediaItem.checkout()` later? 1 paragraph explanation saved as “part3.txt”.

**Part 4: Data Structures as Objects**

In “custom_classes.py”, create a `LibraryCollection` class that acts as a container for your items.

1) Initialize the container with an empty list.
2) Add a method `add_item(item)`: Adds a `MediaItem` (or its subclass) to the list.
3) Add a method `list_available()`: Returns a list of titles that are currently available.

**Practice:** In the “main.py” script, simulate adding at least 3 items of mixed types, check out 2 of them, and print the result of list_available() to simulate available title listing. Push Week3 folder to your Git Repo, committed as “part4”.
**Deliverable:** A screen recording video you simulate adding multiple items and simulate available title listing.; saved as “part4.avi”.

**Part 5: Code review**

You document your code the clean-code way and have a classmate review it through a pull request, PR.

1) On a new branch *docs-and-review*, add a docstring to every class and public method. Add inline comments only where the why is not obvious from the code. Do not comment every line: comments that repeat the code are a clean-code smell.
2) Commit on *docs-and-review* with the message ‘part5’ and push that branch.
3) Partner up with a friend in your class. Open a pull request from *docs-and-review* to main and add your partner as a reviewer.
4) Review your partner's PR. Leave at least three comments, covering one naming or readability point.
5) Respond. Reply to every comment on your own PR, either with a fix commit or a short reason for not changing. Merge, at the end.

**Deliverable:** Link for your merged PR and your github repo.